
Software Requirements Specification

for

Blog Management System API

Version 1.0

Prepared by Md Ismile Hosen Siam

13 January, 2026

Table of Contents

Table of Contents	ii
1. Introduction	1
1.1 Purpose	1
1.2 Project Scope	1
1.3 References	2
2. Overall Description	2
2.1 Product Perspective.....	2
2.2 Product Features	3
2.3 User Classes and Characteristics	4
2.4 Operating Environment.....	4
2.5 Design and Implementation Constraints	5
2.6 Assumptions and Dependencies	6
3. System Requirements	7
3.1 Functional Requirements.....	7
3.2 Non-Functional Requirements	14
3.3 Interface Requirements	15
4. System Architecture	16
4.1 Project Structure	16
4.2 Technology Stack	16
Appendix A: Glossary	15
Appendix B: Analysis Models	18
Use Case Diagram	18
Level 0 Data Flow Diagram	18
Level 1 Data Flow Diagram	19
Entity Relationship Diagram	19

1. Introduction

1.1 Purpose

The purpose of this document is to specify the software requirements for the **Blog Management System API**, Version 1.0. This Software Requirements Specification (SRS) covers the complete backend system of the application, which is designed to allow users to register, log in, and manage blog content through a RESTful API.

The scope of this document includes the core functional modules of the system — **authentication, article management, comment management, and user management** — along with the non-functional aspects that support these modules, including **security, performance, scalability, caching, logging, testing, containerization (Docker), CI/CD, load balancing, and database replication**.

This SRS describes the backend system in its entirety, as the project does not include a client-facing frontend within its current scope; all functionality is exposed and consumed via REST API endpoints.

1.2 Project Scope

The software specified in this document is the **Blog Management System API**, a backend REST API that enables users to register, authenticate, and manage blog content such as articles and comments. The system supports two primary user roles: **registered users**, who can create, update, and delete their own articles and add comments, and **administrators**, who have elevated permissions to manage users and system-wide content.

The primary objective of this system is to provide a secure, scalable, and maintainable backend platform for blog-based content management, following industry-standard practices in software architecture and deployment. Key goals of the system include:

- Providing secure user authentication and role-based access control.
- Enabling efficient creation, retrieval, updating, and deletion (CRUD) of articles and comments.
- Ensuring system reliability and performance through caching, logging, and load balancing.
- Supporting scalability via database replication and containerized deployment (Docker).
- Maintaining code quality and deployment consistency through automated testing and CI/CD pipelines.

This project is developed as a demonstration of backend engineering capability, showcasing practical implementation of authentication systems, RESTful API design, and production-grade deployment practices. It is intended to reflect real-world backend development

standards applicable to scalable content management platforms, and serves as a reference implementation of secure and maintainable API architecture.

No separate vision and scope document exists for this project; this SRS serves as the sole specification of scope for the current release (Version 1.0).

1.3 References

- Project guidelines provided by the client.
- IEEE Recommended Practice for Software Requirements Specifications (IEEE 29148).
- OpenAPI Specification <https://swagger.io/specification/>
- Express.js Documentation: <https://expressjs.com/>
- Node.js Documentation: <https://nodejs.org/docs/>
- MongoDB Documentation: <https://www.mongodb.com/docs/>
- Mongoose Documentation: <https://mongoosejs.com/docs/>
- Docker Documentation <https://docs.docker.com>
- Nodemailer Documentation <https://nodemailer.com/about/>
- JWT Authentication: <https://jwt.io/>
- Redis Documentation: <https://redis.io/docs/>
- Elasticsearch Documentation: <https://www.elastic.co/guide/>
- Winston Logger Documentation: <https://github.com/winstonjs/winston>
- GitHub Actions Documentation: <https://docs.github.com/actions>

2. Overall Description

2.1 Product Perspective

The Blog Management System API is a **new, self-contained backend system** developed independently as a standalone RESTful API. It is not a follow-on to an existing product, nor a replacement for any legacy system currently in use.

The system is designed to function as a **backend subsystem** that can be consumed by external client applications — such as a web frontend, mobile application, or third-party integrations — via well-defined REST API endpoints. While this SRS specifies only the backend components of the system, it is architected to support independent frontend development, since all business logic, data management, and access control are exposed through the API layer.

The system interacts with the following external components:

- **Client Applications** – Any frontend or third-party service that consumes the REST API to perform authentication, content management, and administrative operations.
- **Database Server** – Stores persistent data including users, articles, and comments, with support for replication to ensure scalability and availability.
- **Cache Layer** – Improves performance by reducing redundant database queries for frequently accessed data.
- **CI/CD Pipeline** – Automates testing, building, and deployment of the application to ensure consistent and reliable releases.
- **Load Balancer** – Distributes incoming traffic across multiple instances of the application to maintain performance under load.

2.2 Product Features

The Blog Management System API provides the following major functional groups. Detailed requirements for each are described in Section 3.

Authentication & Authorization

- User registration and login.
- Secure password handling and session/token-based authentication.
- Role-based access control distinguishing between registered users and administrators.

Article Management

- Creation, updating, and deletion of blog articles by their respective authors.
- Retrieval of articles for viewing/listing purposes.
- Enforcement of ownership rules, ensuring users can only modify their own content.

Comment Management

- Addition of comments on articles by registered users.
- Management and moderation of comments where applicable.

User Management

- Administrative capabilities to manage registered user accounts.
- Ability for administrators to oversee and moderate system-wide content.

System Quality & Operations

- Performance optimization through caching mechanisms.
- Logging for monitoring, debugging, and auditing purposes.
- Automated testing to ensure functional reliability.
- Containerized deployment using Docker for consistency across environments.
- Continuous Integration/Continuous Deployment (CI/CD) for streamlined releases.
- Load balancing and database replication to support scalability and high availability.

2.3 User Classes and Characteristics

The Blog Management System API is designed to serve two primary user classes, differentiated by their privilege levels and the scope of functionality available to them.

Registered Users (Standard Users)

- Represent the primary and most frequent user class of the system.
- Able to register, log in, and manage their own blog content.
- Can create, update, and delete their own articles, and add comments on articles.
- Do not have access to administrative functions such as managing other users or moderating system-wide content.
- Assumed to have basic technical familiarity, consistent with typical users of a blogging platform (i.e., no specialized technical expertise required).
- This is the **favored user class**, as the core functionality of the system is built around enabling their content creation and interaction workflows.

Administrators

- Represent a smaller, privileged user class responsible for system oversight.
- Possess all permissions available to registered users, in addition to elevated access for managing user accounts and moderating system content.
- Require a higher level of trust and access control, as their actions affect system-wide data and other users' content.
- Expected to have some familiarity with content moderation and user management responsibilities, though not necessarily deep technical expertise.
- This class is **secondary in frequency of use** compared to standard users, but critical for maintaining system integrity and content quality.

Security and access-control requirements in Section 3 will specify which functions are restricted to Administrators versus available to all Registered Users.

2.4 Operating Environment

The Blog Management System API is built as a backend service intended to run in a containerized, cloud-compatible environment. Its operating environment includes the following components:

Runtime & Framework

- Built on **Node.js**, using the **Express.js** framework for backend application logic and REST API implementation.

Database

- **MongoDB** is used as the primary NoSQL database for persistent data storage, with support for replication (replica sets) to enhance scalability and availability.

Caching Layer

- A caching system (e.g., Redis or equivalent) is used to reduce database load and improve response times for frequently accessed data.

Containerization & Deployment

- The application is packaged and deployed using **Docker**, ensuring consistent behavior across development, testing, and production environments.
- Deployment is supported by a **CI/CD pipeline**, automating build, test, and release processes.

Infrastructure

- The system is designed to operate behind a **load balancer**, distributing incoming traffic across multiple application instances to maintain performance and availability under load.
- Compatible with standard cloud hosting environments (e.g., AWS, DigitalOcean, or similar Linux-based server environments) that support Docker containers.

Operating System

- The application is platform-independent at the code level (Node.js-based) but is intended to be deployed on **Linux-based server environments**, which is standard for Docker-based production deployments.

2.5 Design and Implementation Constraints

Technology Stack: The system must be implemented using **Node.js** with the **Express.js** framework for the backend, and **MongoDB** as the database, as these are the fixed technology choices for this project.

RESTful Architecture: All client-server communication must follow REST API conventions, using standard HTTP methods (GET, POST, PUT, DELETE) and status codes.

Authentication Protocol: User authentication must implement industry-standard practices (e.g., JWT-based token authentication) to ensure secure access control.

Containerization: The application must be deployable via **Docker**, requiring all components to be compatible with containerized environments.

Database Design: As MongoDB is a NoSQL database, data modeling must follow document-oriented design principles rather than relational schema conventions.

Security Requirements: Sensitive data such as user passwords must be encrypted/hashed (e.g., using bcrypt) and never stored or transmitted in plain text.

Communication Protocol: All API communication must occur over HTTPS in production environments to ensure data security in transit.

Testing Standards: The system must include automated tests to validate functionality before deployment, as part of the CI/CD pipeline.

Deployment Environment: The system is designed for Linux-based cloud hosting environments compatible with Docker and standard CI/CD tooling.

2.6 Assumptions and Dependencies

Assumptions

- It is assumed that client applications (web/mobile frontend) consuming this API will be developed independently and are not part of this SRS's scope.
- It is assumed that the hosting environment will provide adequate resources (CPU, memory, storage) to support Docker containers, database replication, and load balancing as the system scales.
- It is assumed that users will access the system over a stable internet connection, as the API does not account for offline functionality.
- It is assumed that MongoDB's replication and scalability features will be sufficient to meet the system's performance goals without requiring a shift to a different database technology.

Dependencies

- The system depends on **MongoDB** as a third-party database service, and any changes to its API, licensing, or availability could impact the project.
- The system depends on external caching solutions (e.g., Redis) for performance optimization; unavailability or misconfiguration of this component could affect response times.
- The project depends on third-party libraries and packages within the Node.js/Express ecosystem (e.g., authentication middleware, validation libraries); changes or deprecations in these dependencies could require code updates.
- The CI/CD pipeline depends on the availability of the chosen CI/CD platform (e.g., GitHub Actions, GitLab CI, Jenkins) for automated testing and deployment.
- The system's scalability features (load balancing, replication) depend on the hosting infrastructure supporting these capabilities.

3. System Requirements

3.1 Functional Requirements

3.1.1 User Registration

- **Description:** Allow new users to create an account.
- **Input:** Name (string), email (string, valid format), password (string, minimum 8 characters).
- **Output:** Success message with user info or error.
- **Process:**
 - Validate input data.
 - Hash the password using bcrypt.
 - Store user data in the database.
 - Send verification email.
- **Constraints:** Email must be unique.

3.1.2 Email Verification

- **Description:** Verify user email to activate account.
- **Input:** Verification token (string).
- **Output:** Success message with JWT token or error message.
- **Process:**
 - Validate token.
 - Activate user account.
- **Constraints:** Token must be valid and not expired.

3.1.3 Resend Verification Email

- **Description:** Resend email verification link.
- **Input:** Email (string).
- **Output:** Success or error message.
- **Process:**
 - Validate email.
 - Generate new verification token.
 - Send email.
- **Constraints:** Only unverified users allowed.

3.1.4 User Login

- **Description:** Authenticate user using email and password.
- **Input:** Email (string), password (string).
- **Output:** Access token and refresh token or error message.

- **Process:**
 - Validate input data.
 - Verify email and password.
 - Generate JWT access token and refresh token.
- **Constraints:** Only verified and active users can log in.

3.1.5 Token Refresh

- **Description:** Generate a new access token using a valid refresh token.
- **Input:** Refresh token (HttpOnly cookie).
- **Output:** New access token or error message.
- **Process:**
 - Validate refresh token.
 - Generate new access token.
- **Constraints:** Refresh token must be valid and not expired.

3.1.6 Password Reset Request

- **Description:** Allow users to request a password reset link via email.
- **Input:** Registered email address (string).
- **Output:** Success message or error.
- **Process:**
 - Validate email.
 - Generate reset token.
 - Send password reset link via email.
- **Constraints:** Email must be registered in the system.

3.1.7 Password Reset Confirmation

- **Description:** Allow users to set a new password using a reset token.
- **Input:** Reset token (string), new password (string).
- **Output:** Success message or error.
- **Process:**
 - Validate reset token.
 - Hash new password.
 - Update password in database.
 - Invalidate reset token.
- **Constraints:** Token must be valid and not expired.

3.1.8 User Logout

- **Description:** Log out user and invalidate authentication session.
- **Input:** Refresh token (cookie).
- **Output:** Success message.

- **Process:**
 - Receive logout request.
 - Invalidate refresh token.
- **Constraints:** User must be authenticated.

3.1.9 Change Password

- **Description:** Allow users to update their account password.
- **Input:** User ID, old and new password (string).
- **Output:** Success message or error.
- **Process:**
 - Validate input data.
 - Verify old password.
 - Hash new password using bcrypt.
 - Update password in database.
- **Constraints:** User must be authenticated.

3.1.10 Initial Admin Setup

- **Description:** Create the first administrator account for the system.
- **Input:** Name (string), email (string), password (string).
- **Output:** Success message or error.
- **Process:**
 - Check if admin exists.
 - Create admin account.
 - Disable setup endpoint after first admin creation.
- **Constraints:** Can only be used once (first-time setup only).

3.1.11 Create Article

- **Description:** Allow authenticated users to create a new article.
- **Input:** Title (string), body (string), cover image (url).
- **Output:** Created article or error.
- **Process:**
 - Validate input data.
 - Verify user authentication.
 - Save article in database.
- **Constraints:** Only authenticated users can create articles.

3.1.12 View published Articles

- **Description:** Allow users to retrieve a list of articles.
- **Input:** Pagination parameters (page, limit), sorting (asc, desc, createdAt, updatedAt, title), searching (title).

- **Output:** List of articles or empty response.
- **Process:**
 - Fetch articles.
 - Apply sorting, searching, pagination.
- **Constraints:** Only published articles returned.

3.1.13 View Article Details

- **Description:** Retrieve single article.
- **Input:** Article ID, Expand options.
- **Output:** Article data or error.
- **Process:**
 - Retrieve articles from database.
 - Include related data if requested.
- **Constraints:** Article must exist. Public sees only published articles.

3.1.14 View Article Author

- **Description:** Retrieve author of article.
- **Input:** Article ID.
- **Output:** Author details or error.
- **Process:**
 - Fetch article.
 - Return author info.
- **Constraints:** Article must exist.

3.1.15 Update Article

- **Description:** Allow users to update their own articles.
- **Input:** Article ID, updated fields (title, body, cover image).
- **Output:** Success message or error.
- **Process:**
 - Validate input data.
 - Verify user authorization.
 - Update article in database.
- **Constraints:** Only the article owner can update.

3.1.16 Delete Article

- **Description:** Allow users to delete their own articles.
- **Input:** Article ID.
- **Output:** Success message or error.
- **Process:**
 - Verify user authorization.

- Delete article from database.
- **Constraints:** Only the article owner or admin can delete.

3.1.17 Manage Article Status

- **Description:** Publish or draft the article.
- **Input:** Article ID, Status.
- **Output:** Success message or error.
- **Process:**
 - Validate status.
 - Update article status.
- **Constraints:** Only admin can update the status of an article.

3.1.18 View All Articles (Admin)

- **Description:** Retrieve all articles.
- **Input:** Page, Limit, Filters.
- **Output:** List of articles.
- **Process:**
 - Authenticate admin.
 - Fetch all articles.
- **Constraints:** Only admin can access this route.

3.1.19 Create Comment

- **Description:** Allow authenticated users to add comments to an article.
- **Input:** Article ID, comment text (string).
- **Output:** Created comment or error message.
- **Process:**
 - Validate input data.
 - Verify user authentication.
 - Store comment in database.
- **Constraints:** User must be authenticated. Article must exist.

3.1.20 View Article Comments

- **Description:** Allow users to view comments of a specific article.
- **Input:** Article ID, pagination.
- **Output:** List of comments or empty response.
- **Process:**
 - Fetch comments
 - Apply pagination
- **Constraints:** Only public comments visible. Article must exist.

3.1.21 Update Comment

- **Description:** Allow users to update their own comments.
- **Input:** Comment ID, updated text (string).
- **Output:** Success message or error.
- **Process:**
 - Validate input data.
 - Verify user authorization.
 - Update comment in database.
- **Constraints:** Only comment owner can update.

3.1.22 Delete Comment

- **Description:** Allow users to delete their own comments.
- **Input:** Comment ID.
- **Output:** Success message or error.
- **Process:**
 - Verify user authorization.
 - Delete comment from database.
- **Constraints:** Only comment owner or admin can delete.

3.1.23 Manage Comment Visibility

- **Description:** Hide or show comments.
- **Input:** Comment ID, Status.
- **Output:** Success message or error.
- **Process:**
 - Verify authorization.
 - Update comment status
- **Constraints:** Only admin can update status.

3.1.24 View All Comments (Admin)

- **Description:** Retrieve all comments.
- **Input:** Filters, Pagination.
- **Output:** List of comments or error.
- **Process:**
 - Verify admin authorization.
 - Fetch comments.
- **Constraints:** Only admin can access this route.

3.1.25 Create User

- **Description:** Allow administrators to create new user accounts.
- **Input:** Name (string), email (string), password (string).

- **Output:** Created user or error message.
- **Process:**
 - Validate input data.
 - Verify admin authorization.
 - Create user account in database.
- **Constraints:** Only administrators can create users. Email must be unique.

3.1.26 View Users

- **Description:** Allow administrators to view all user accounts.
- **Input:** Filters, Pagination.
- **Output:** List of users or empty response.
- **Process:**
 - Verify admin authorization.
 - Retrieve users from database.
- **Constraints:** Only administrators can access user list.

3.1.27 View User Details

- **Description:** Retrieve single user.
- **Input:** User ID, Expand.
- **Output:** User data.
- **Process:**
 - Verify authorization.
 - Fetch user
- **Constraints:** User id must exist.

3.1.28 Update User Information

- **Description:** Allow administrators to update user details.
- **Input:** Name (string), role (string), status (string).
- **Output:** Success message or error.
- **Process:**
 - Validate input data.
 - Verify admin authorization.
 - Update user information in database.
- **Constraints:** Only administrators can update user data.

3.1.29 Delete User

- **Description:** Allow administrators to delete user accounts.
- **Input:** User ID.
- **Output:** Success message or error.
- **Process:**
 - Verify admin authorization.

- Delete user from database.
- **Constraints:** Only administrators can delete.

3.2 Non-Functional Requirements

3.2.1 Performance

- **Response Time:** API responses should return within 1-2 seconds under normal load.
- **Scalability:** The system should handle up to 50-100 concurrent users.
- **Throughput:** The system should support at least 300–600 API requests per minute without degradation.

3.2.2 Security

- **Authentication:** JWT for authentication for APIs.
- **Data Protection:** Hash passwords with bcrypt. Encrypt sensitive data in transit (HTTPS).
- **Authorization:** Role-based access control to restrict access to resources based on user roles.
- **Rate limiting:** 5 request per minute for login and reset password functionality.

3.2.3 Reliability

- **Uptime:** 99.9% availability for APIs.
- **Data Integrity:** Ensure consistency of blog posts, comments, and user data during create/update/delete operations.
- **Error Handling:** Proper error handling and logging mechanisms should be implemented to prevent system crashes.

3.2.4 Usability

- **API Design:** APIs should follow RESTful principles with clear and consistent endpoint naming.
- **API Documentation:** Provide API documentation using Swagger/OpenAPI.
- **Developer Experience:** Include clear request/response examples and error messages for easier integration.

3.2.5 Maintainability

- **Modularity:** Follow a layered architecture.
- **Code Quality:** Enforce ESLint and Prettier for consistent code formatting and quality.
- **Testing:** Ensure at least 70–80% unit test coverage for core business logic.

- **Version Control:** Maintain clean Git workflow with proper commit structure.

3.2.6 Portability

- **Deployment:** The system should be deployable on any cloud platform supporting Node.js (e.g., Docker-based environments).
- **Database:** Compatible with MongoDB local and cloud instances.

3.3 Interface Requirements

3.3.1 REST API

- **Endpoints (/api/v1)**
 - POST /auth/sign-up: Register a new user
 - POST /auth/setup-admin: Create initial admin account
 - POST /auth/sign-in: Log in and receive access token
 - POST /auth/refresh: Refresh access token
 - POST /auth/logout: Log out user
 - GET /auth/verify-email/{token}: Verify email address
 - POST /auth/resend-verification: Resend verification email
 - POST /auth/forgot-password: Request password reset
 - PATCH /auth/reset-password/{token}: Reset password
 - POST /articles: Create a new article
 - GET /articles: List published articles
 - GET /articles/admin/all: List all articles (admin)
 - GET /articles/{id}: Get article details
 - PUT /articles/{id}: Create or fully update article
 - PATCH /articles/{id}: Partially update article
 - DELETE /articles/{id}: Delete article
 - GET /articles/{id}/author: Get article author
 - GET /articles/{id}/comments: List article comments
 - POST /articles/{id}/comments: Create comment for article
 - GET /comments: List all comments (admin)
 - POST /comments: Create a comment
 - PATCH /comments/{id}: Update comment
 - DELETE /comments/{id}: Delete comment
 - POST /users: Create a user (admin)
 - GET /users: List all users (admin)
 - GET /users/{id}: Get user details
 - PATCH /users/{id}: Update user
 - DELETE /users/{id}: Delete user
 - PATCH /users/{id}/change-password: Change user password

4. System Architecture

4.1 Project Structure

The system follows a layered architecture where all components are organized within a single backend application.

- **server/**: Root backend application directory
 - **src/**: Contains the core source code
 - **api/**: Handles request processing and controllers
 - **routes/**: Defines API routes
 - **model/**: Contains database models (Mongoose schemas)
 - **db/**: Database connection configuration
 - **middleware/**: Custom middleware (auth, error handling)
 - **config/**: Application configuration
 - **lib/**: Shared services (e.g., token, email)
 - **utils/**: Helper functions
 - **app.js**: Express app configuration
 - **index.js**: Application entry point
 - **.env**: Environment variables
 - **swagger.yaml**: API documentation
 - **package.json** → Dependencies and scripts
 - **.eslintrc.json** → ESLint configuration

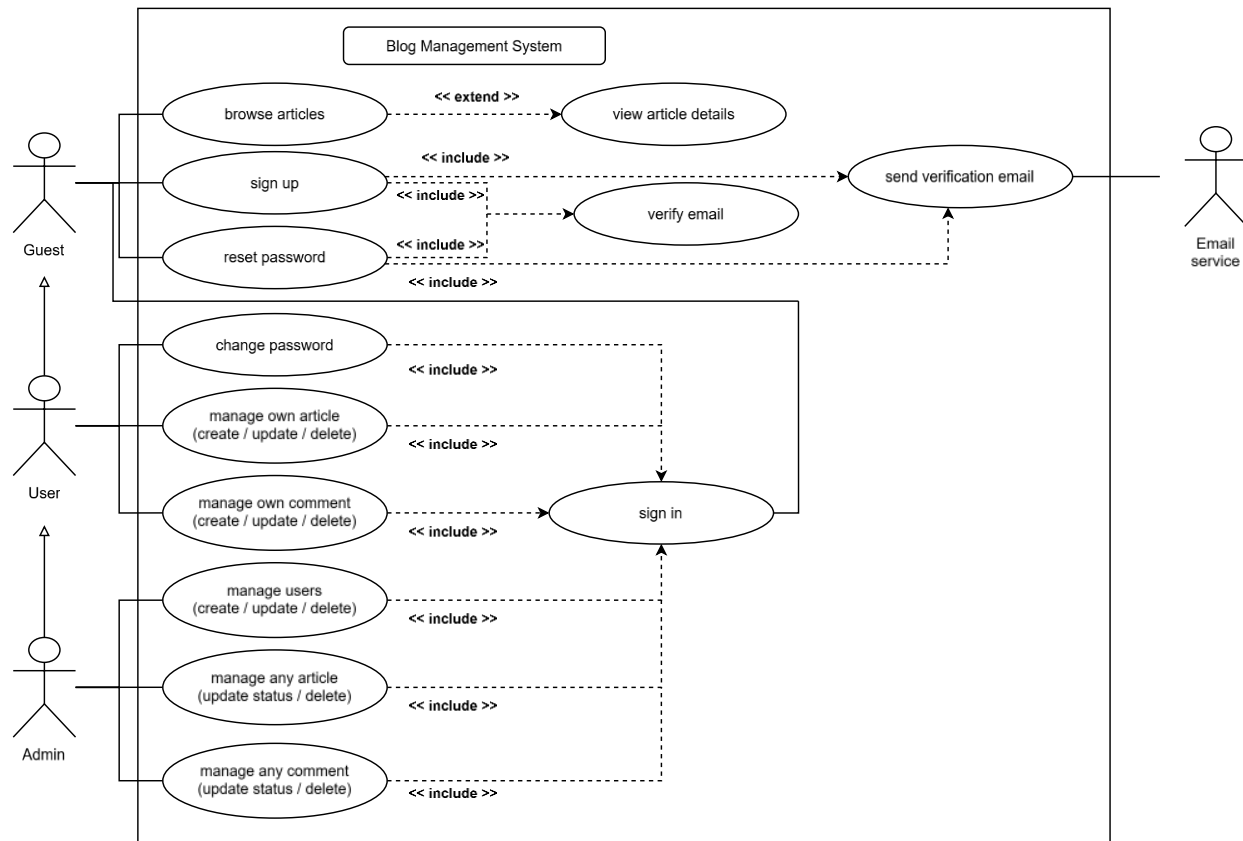
4.2 Technology Stack

- **Backend**: Node.js, Express.js.
- **Database**: MongoDB, Mongoose (ODM).
- **Authentication & Security**: JWT, bcrypt, HttpOnly Cookies.
- **Email Service**: Nodemailer with SMTP.
- **API Documentation**: OpenAPI (Swagger YAML).
- **Development Tools**: ESLint, Yarn, Prettier
- **Deployment Tools**: Docker and CI/CD tools for automated build and deployment.
- **Caching**: Redis.
- **Logging**: Winston for structured application logging; **Elasticsearch (ELK stack)** for centralized log aggregation.

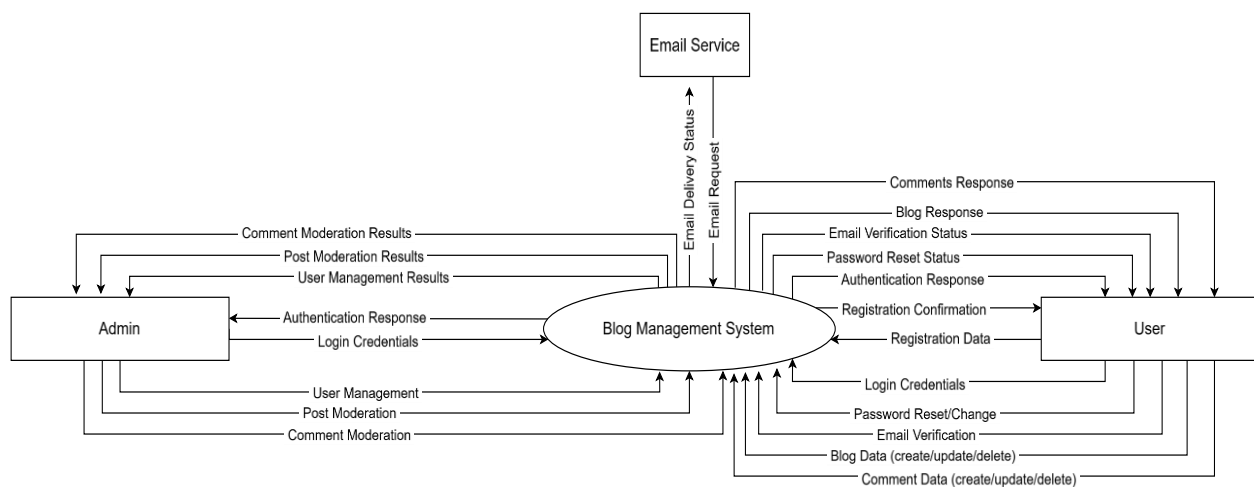
Appendix A: Glossary

- **API:** Application Programming Interface.
- **JWT:** JSON Web Token used for authentication.
- **CI/CD:** Continuous Integration / Continuous Deployment.
- **SMTP:** Simple Mail Transfer Protocol used for sending emails.
- **REST:** Representational State Transfer architecture for web services.
- **RBAC:** Role-Based Access Control.

Appendix B: Analysis Models

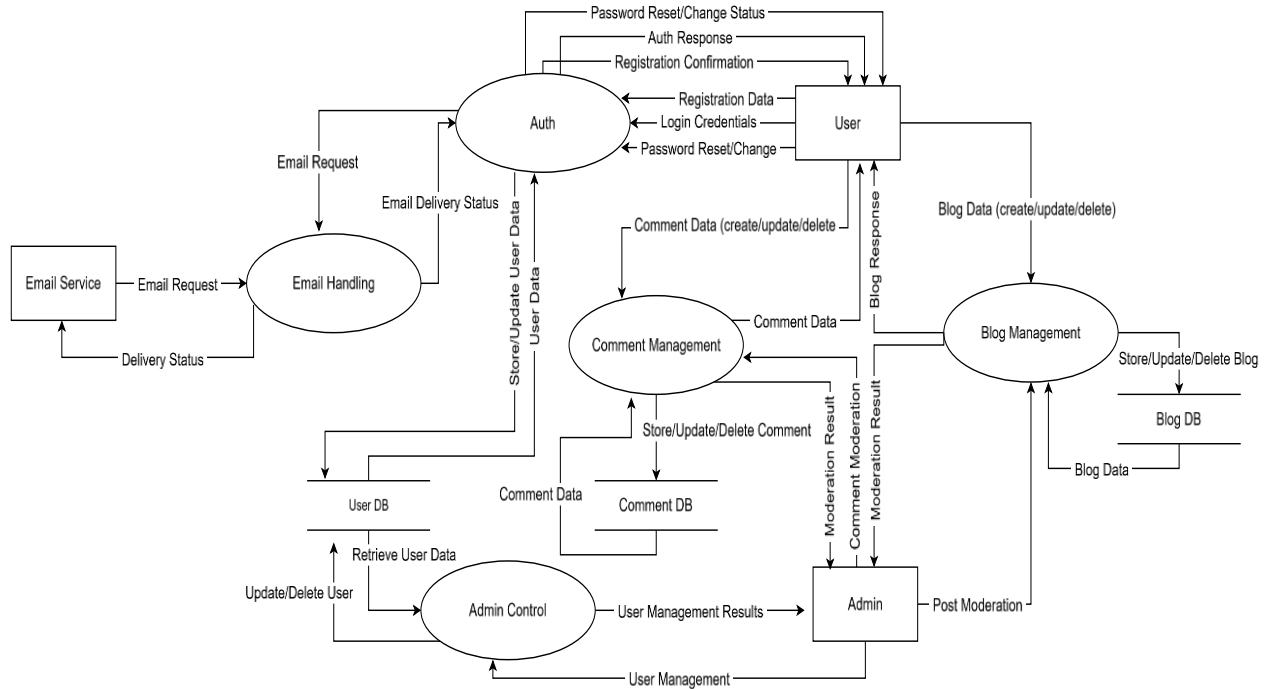


Use Case Diagram

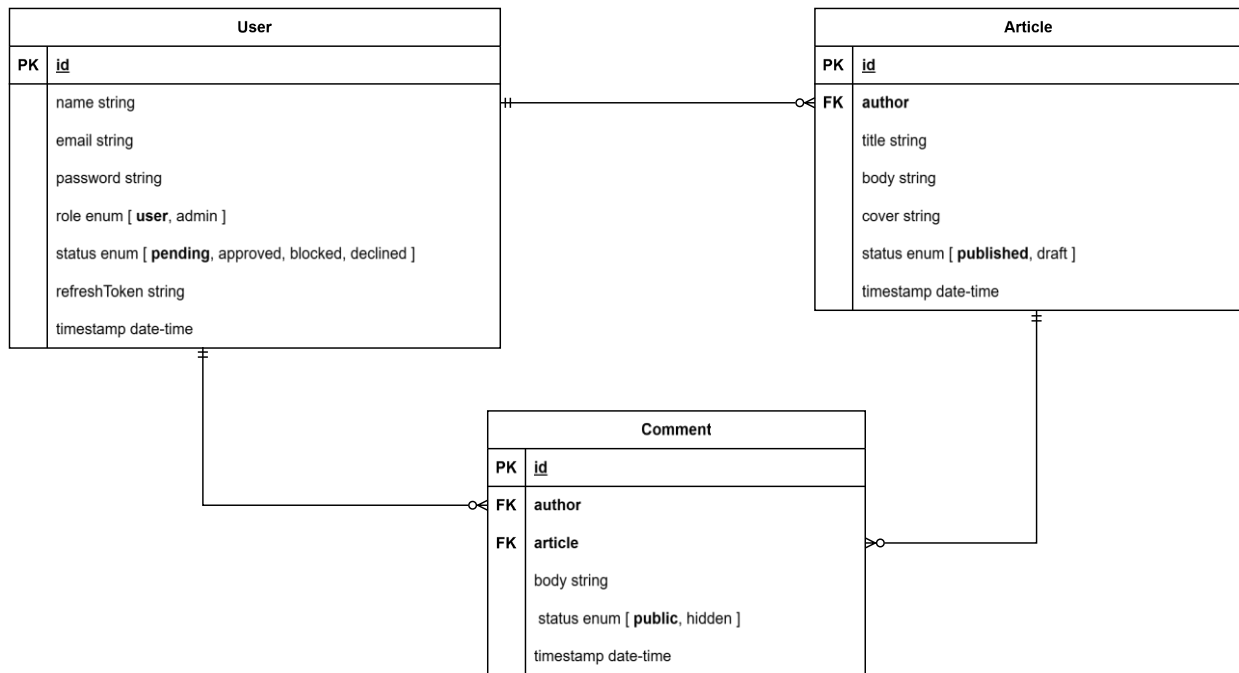


Level 0 Data Flow Diagram

Software Requirements Specification for Blog Management System API



Level 1 Data Flow Diagram



Entity Relationship Diagram